

MOCK EXAM

practice

Q. Fix the code to find sum of the factorial of all odd numbers

```
import java.io.*;
import java.util.*;
import java.lang.Math;

public class Solution {
    public static long factorial(int n){
        long fact = 1;
        for(int i=1; i<=n; i++){
            fact*=i;
        }
        return fact;
    }
    public static long buggySumOfOddFactorials(int n, List<Integer> arr) {
        // Fix the code here
        long sum = 0;
        for (int i = 0; i < arr.size(); i++) {
            if (arr.get(i) % 2 == 1) {
                sum += factorial(arr.get(i));
            }
        }
        return sum;
    }
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);

        int n = Integer.parseInt(scan.nextLine().trim());

        List<Integer> arr = new ArrayList<>(n);
        for(int j=0; j<n; j++) {
            arr.add(Integer.parseInt(scan.nextLine().trim()));
        }

        long result = buggySumOfOddFactorials(n, arr);

        System.out.println(result);
    }
}
```

Javascript question

Q. find the final status of the package

```
function solve(statuses) {  
    const statusList = statuses.split(';');  
    return statusList[statusList.length-1];  
    // Write your code here  
  
}
```

```
const statuses = gets();
```

```
const result = solve(statuses);
```

```
print(result)
```

SQL Question

Q. Course Enrollment

-- Enter your query here

-- Note: MySQL queries are case-sensitive. To ensure correctness of the code, please follow the same standard.

```
SELECT c.course_name,  
       COUNT(e.student_id) AS student_count  
FROM courses c  
LEFT JOIN  
       enrollments e ON c.course_id=e.course_id  
GROUP BY  
       c.course_id,c.course_name  
ORDER BY  
       c.course_id;
```

Spring Security Question

Q. Student Management Microservice

StudentController.java

```
package com.student.api.controller;
```

```
import com.student.api.domain.Student;  
import org.springframework.dao.EmptyResultDataAccessException;  
import org.springframework.http.ResponseEntity;  
import org.springframework.jdbc.core.namedparam.MapSqlParameterSource;  
import org.springframework.jdbc.core.namedparam.NamedParameterJdbcTemplate;  
import org.springframework.jdbc.core.namedparam.SqlParameterSource;  
import org.springframework.web.bind.annotation.*;
```

```

import java.util.List;

/**
 * REST controller for managing student system process. Use {@link StudentRowMapper} to
 * map database rows to Student entity object.
 */

@RestController
@RequestMapping("/api/v1")
public class StudentController {

    // use JdbcTemplate to query for students against database
    private final NamedParameterJdbcTemplate jdbcTemplate;

    public StudentController(NamedParameterJdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    /**
     * {@code GET /students} : get all the Students.
     *
     * @return the {@link ResponseEntity} with status {@code 200 (OK)} and the list
     * of students in body.
     */
    @GetMapping("/students")
    public ResponseEntity<List<Student>> getAllStudents() {

        List<Student> students = jdbcTemplate.query("SELECT * FROM student", new
        StudentRowMapper());
        return ResponseEntity.ok().body(students);
        //return ResponseEntity.ok().body(null);
    }

    /**
     * {@code GET /students/{id}} : get the "id" Student.
     *
     * @param id the id of the student to retrieve.
     * @return the {@link ResponseEntity} with status {@code 200 (OK)} and with body
     * the student, or if does not exist, return with status "noContent".
     */
    // @GetMapping("/students/{id}")
    // public ResponseEntity<Student> getStudent(@PathVariable Long id) {
    /**
     * {@code GET /students/{id}} : get the "id" Student.
     *
     * @param id the id of the student to retrieve.
     * @return the {@link ResponseEntity} with status {@code 200 (OK)} and with body

```

```

* the student, or if does not exist, return with status "noContent".
*/

@GetMapping("/students/{id}")
public ResponseEntity<Student> getStudent(@PathVariable Long id) {
    try {
        Student student = jdbcTemplate.queryForObject("SELECT * FROM student WHERE
id = :id", new MapSqlParameterSource("id", id), new StudentRowMapper());
        return ResponseEntity.ok().body(student);
    } catch (EmptyResultDataAccessException e) {
        return ResponseEntity.noContent().build();
    }
    // return ResponseEntity.ok().body(null);
}

/**
 * {@code POST /student} : Create a new student.
 *
 * @param student the student to create.
 * @return the {@link ResponseEntity} with status {@code 200 (OK)} and with
 * body the new student
 */
@PostMapping("/students")
public ResponseEntity<Void> createStudent(@RequestBody Student student) {
    int rowsAffected = jdbcTemplate.update("INSERT INTO student (id, name) VALUES
(:id, :name)",
        new MapSqlParameterSource()
            .addValue("id", student.getId())
            .addValue("name", student.getName());
    return rowsAffected > 0 ? ResponseEntity.ok().build() :
ResponseEntity.noContent().build();
}

// @PostMapping("/students")
// public ResponseEntity<Void> createStudent(@RequestBody Student student) {

//     return ResponseEntity.ok().build();
// }

/**
 * {@code PUT /student} : Updates an existing student.
 *
 * @param student the student to update.
 * @return the {@link ResponseEntity} with status {@code 200 (OK)} and with body
 * the updated student.
 */

```

```

    @PutMapping("/students")
    public ResponseEntity<Void> updateStudent(@RequestBody Student student) {
        int rowsAffected = jdbcTemplate.update("UPDATE student SET name = :name
WHERE id = :id",
        new MapSqlParameterSource()
            .addValue("id", student.getId())
            .addValue("name", student.getName()));
        return rowsAffected > 0 ? ResponseEntity.ok().build() :
ResponseEntity.noContent().build();
    }

    /**
     * {@code DELETE /student/{id}} : delete the "id" student.
     *
     * @param id the id of the student to delete.
     * @return the {@link ResponseEntity} with status {@code 200 (OK)}.
     */
    // @DeleteMapping("/students/{id}")
    // public ResponseEntity<Void> deleteStudent(@PathVariable Long id) {

        /**
         * {@code DELETE /student/{id}} : delete the "id" student.
         *
         * @param id the id of the student to delete.
         * @return the {@link ResponseEntity} with status {@code 200 (OK)}.
         */

        @DeleteMapping("/students/{id}")
        public ResponseEntity<Void> deleteStudent(@PathVariable Long id) {
            int rowsAffected = jdbcTemplate.update("DELETE FROM student WHERE id = :id",
new MapSqlParameterSource("id", id));
            return rowsAffected > 0 ? ResponseEntity.ok().build() :
ResponseEntity.noContent().build();
        }
    }

```

```

// jdbcTemplate.update("DELETE FROM student WHERE id =?", id);

```

StudentRowMapper.java

```
package com.student.api.controller;
```

```
import com.student.api.domain.Student;
```

```
import org.springframework.jdbc.core.RowMapper;
```

```
import java.sql.ResultSet;
```

```
import java.sql.SQLException;
```

```
public class StudentRowMapper implements RowMapper<Student> {
```

```
    /**
```

```
     *
```

```
     * @param rs Database ResultSet object. Get database data with the column names "ID"
and "NAME". Remember, "ID" column is Long data type and "NAME" column is String data
type.
```

```
     * @param rowNum If you get data with column names as described above, you don't
need to use rowNum parameter
```

```
     * @return Student object with the mapped values from database
```

```
    */
```

```
    @Override
```

```
    public Student mapRow(ResultSet rs, int rowNum) throws SQLException {
```

```
        Long id = rs.getLong("ID");
```

```
        String name = rs.getString("NAME");
```

```
        return new Student(id, name);
```

```
    }
```

```
}
```


REACT QUESTION

Q. Build a Sales Dashboard Application

Dashboard.jsx

```
import axios from "axios";
```

```
import React, { useEffect, useState } from "react";
```

```
import "../Dashboard.css";
```

```
import { calculateTotalSales, calculateTotalCashSale, calculateTotalCreditSale,
calculateBuyerWithMostSale } from './Reports';
```

```
function Dashboard(){
```

```
    const App=()=>{
```

```
        const[data, setData]=useState([]);
```

```
        useEffect(()=>{
```

```

    (async())=>{
      const result=await axios.get('/sales.json');
      setData(result.data);
    })();
  },[])

  return (
    <div className="dashboard">
      <div className="card">
        <h2>Total Sales</h2>
        <p>{calculateTotalSales(data)}</p>
      </div>
      <div className="card">
        <h2>Total Cash Sales</h2>
        <p>{calculateTotalCashSale(data)}</p>
      </div>
      <div className="card">
        <h2>Total Credit Sales</h2>
        <p>{calculateTotalCreditSale(data)}</p>
      </div>
      <div className="card">
        <h2>Buyer with Most Sales</h2>
        <p>{calculateBuyerWithMostSale(data).buyerName}</p>
        <p>{calculateBuyerWithMostSale(data).saleTotal}</p>
      </div>
    </div>
  );
}
}
export default Dashboard;

```

Reports.js

```

import axios from "axios";

export const getSalesData = async () => {
  let { data } = await axios.get(`/sales.json`);
  return data;
};

export const calculateTotalSales = (sales) => {
  return sales.reduce((total, sale) => total+sale.saleTotal, 0);
};

export const calculateTotalCashSale = (sales) => {
  return sales.filter(sale=>sale.creditCard===false)

```

```

    .reduce((total,sale)=>total+sale.saleTotal,0);

};

export const calculateTotalCreditSale = (sales) => {
  return sales.filter(sale=>sale.creditCard===true)
    .reduce((total, sale)=>total+sale.saleTotal,0);
};

export const calculateBuyerWithMostSale = (sales) => {
  const buyerMap={};
  for(const sale of sales){
    if(!buyerMap[sale.buyerName]){
      buyerMap[sale.buyerName]=0;
    }
    buyerMap[sale.buyerName]+=sale.saleTotal;
  }
  let maxBuyer=null;
  let maxTotal=0;
  for(const [buyer,total] of Object.entries(buyerMap)){
    if(total>maxTotal){
      maxBuyer=buyer;
      maxTotal=total;
    }
  }
  return {
    buyerName:maxBuyer,
    saleTotal:maxTotal
  };
};

```

HTML/CSS/JS Question

Q. Bank Management System Form

index.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Transaction Filter</title>
  <style>
    body {
      background-color: gray;
    }
  </style>

```



```
</head>
<body>
  <div>
    <select id="transactionType">
      <option value="all">All</option>
      <option value="deposit">Deposits</option>
      <option value="withdrawal">Withdrawals</option>
    </select>
    <ul id="transactionList"></ul>
  </div>
</body>

</html>
```

index.css

```
body {
  background-color: gray;
}
```

index.js

```
const transactions = [
  { type: "deposit", amount: 100 },
  { type: "withdrawal", amount: 50 },
  { type: "deposit", amount: 200 },
  { type: "withdrawal", amount: 30 },
  { type: "deposit", amount: 150 }
];

function filterTransactions(type, container) {
  container.innerHTML = ""; // Clear previous entries

  const filtered = type === "all"
    ? transactions
    : transactions.filter(txn => txn.type === type);

  filtered.forEach(txn => {
    const li = document.createElement("li");
    li.textContent = `${txn.type.toUpperCase()}: $${txn.amount}`;
    container.appendChild(li);
  });
}

module.exports = filterTransactions;
```

FINAL EXAM

total practice

```
import java.io.*;

import java.util.*;

import java.math.BigInteger;

public class Solution {

    // Method to compute factorial of a number using BigInteger

    public static BigInteger factorial(int num) {

        BigInteger fact = BigInteger.ONE;

        for (int i = 2; i <= num; i++) {

            fact = fact.multiply(BigInteger.valueOf(i));

        }

        return fact;

    }

    public static BigInteger sumOfOddFactorials(int n, List<Integer> arr) {

        BigInteger sum = BigInteger.ZERO;

        for (int i = 0; i < n; i++) {

            int val = arr.get(i);

            if (val % 2 != 0) { // Check if odd

                sum = sum.add(factorial(val));

            }

        }

    }

}
```

```

        return sum;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        List<Integer> arr = new ArrayList<>();

        for (int i = 0; i < n; i++) {

            arr.add(sc.nextInt());

        }

        BigInteger result = sumOfOddFactorials(n, arr);

        System.out.println(result);

    }
}

```

=====

js 2

=====

```

function solve(statuses) {

    // Write your code here

    const arr = statuses.split(';')

    return arr[arr.length-1]

}

const statuses = gets();

const result = solve(statuses);

print(result)

```

=====

sql?

=====

SELECT

c.course_name,

COUNT(e.student_id) AS num_students

FROM

courses c

LEFT JOIN

enrollments e ON c.course_id = e.course_id

GROUP BY

c.course_id, c.course_name;

crud book store

----- service

package com.wecreateproblems.crudcollectionapp.service;

import com.wecreateproblems.crudcollectionapp.entity.Book;

import org.springframework.stereotype.Service;

import java.util.ArrayList;

@Service

```

public class BookService {

    private final Map<Long, Book> books = new HashMap<>();

    public void createBook(Book book) {

        // add a book to map

        books.put(book.getId(), book);

    }

    public List<Book> getAllBooks() {

        // return list of books from map

        return new ArrayList<>(books.values());

    }

}

```

----- controller

```

package com.wecreateproblems.crudcollectionapp.controller;

import com.wecreateproblems.crudcollectionapp.entity.Book;
import com.wecreateproblems.crudcollectionapp.service.BookService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/books")

public class BookController {

    @Autowired

```

```

private final BookService bookService;

@Autowired

public BookController(BookService bookService) {

    this.bookService = bookService;

}

@PostMapping

public void createBook(@RequestBody Book book) {

    // add book to collection

    bookService.createBook(book);

}

@GetMapping

public List<Book> getAllBooks() {

    // return all books from collection

    return bookService.getAllBooks();

}

}

```

=====

security:

```
package com.wecp.w3day5task1.entity;
```

```
import javax.persistence.*;
```

@Entity

@Table(name = "users")

public class User {

 @Id

 @GeneratedValue(strategy = GenerationType.IDENTITY)

 private Long id;

 @Column(unique = true, nullable = false)

 private String username;

 @Column(nullable = false)

 private String password;

 // Roles stored as a comma-separated string, e.g. "USER" or "ADMIN"

 @Column(nullable = false)

 private String roles;

 // Getters and Setters

 public Long getId() {

 return id;

 }

 public void setId(Long id) {

 this.id = id;

 }

 public String getUsername() {

 return username;

 }

 public void setUsername(String username) {

```

        this.username = username;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getRoles() {
        return roles;
    }

    public void setRoles(String roles) {
        this.roles = roles;
    }
}

package com.wecp.w3day5task1.repository;

import com.wecp.w3day5task1.entity.User;

import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface UserRepository extends JpaRepository<User, Long> {

    Optional<User> findByUsername(String username);
}

package com.wecp.w3day5task1.service;

import com.wecp.w3day5task1.entity.User;

```



```

import com.wecp.w3day5task1.repository.UserRepository;

import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.security.core.authority.AuthorityUtils;

import org.springframework.security.core.userdetails.UserDetails;

import org.springframework.security.core.userdetails.UserDetailsService;

import org.springframework.security.core.userdetails.UsernameNotFoundException;

import org.springframework.stereotype.Service;

@Service

public class CustomUserDetailsService implements UserDetailsService {

    @Autowired

    private UserRepository userRepository;

    @Override

    public UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException {

        User user = userRepository.findByUsername(username)

            .orElseThrow(() -> new UsernameNotFoundException("User not found: " + username));

        return new org.springframework.security.core.userdetails.User(

            user.getUsername(),

            user.getPassword(),

            AuthorityUtils.commaSeparatedStringToAuthorityList(user.getRoles())

        );

    }

}

package com.wecp.w3day5task1.config;

import com.wecp.w3day5task1.service.CustomUserDetailsService;

```

```

import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.context.annotation.Bean;

import org.springframework.context.annotation.Configuration;

import
org.springframework.security.config.annotation.authentication.builders.AuthenticationManag
erBuilder;

import org.springframework.security.config.annotation.web.builders.HttpSecurity;

import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;

import
org.springframework.security.config.annotation.web.configuration.WebSecurityConfigurerAd
apter;

import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;

import org.springframework.security.crypto.password.PasswordEncoder;

@Configuration

@EnableWebSecurity

public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Autowired

    private CustomUserDetailsService userDetailsService;

    @Override

    protected void configure(HttpSecurity http) throws Exception {

        http

            .httpBasic() // Enable HTTP Basic Authentication

            .and()

            .authorizeRequests()

                .antMatchers("/admin").hasRole("ADMIN") // Only ADMIN role can access /admin

```

```

        .anyRequest().authenticated() // All other requests require authentication

        .and()

        .csrf().disable(); // Disable CSRF for simplicity (not recommended for production)
    }

    @Override

    protected void configure(AuthenticationManagerBuilder auth) throws Exception {

        // Configure in-memory authentication with two users

        auth.inMemoryAuthentication()

            .withUser("user")

            .password(passwordEncoder().encode("userpass"))

            .roles("USER")

        .and()

            .withUser("admin")

            .password(passwordEncoder().encode("adminpass"))

            .roles("ADMIN");

        // Also enable authentication with UserDetailsService (optional)

        auth.userDetailsService(userDetailsService).passwordEncoder(passwordEncoder());
    }

    @Bean

    public PasswordEncoder passwordEncoder() {

        return new BCryptPasswordEncoder();

    }

}

package com.wecp.w3day5task1.controller;

```

```

import org.springframework.web.bind.annotation.GetMapping;

import org.springframework.web.bind.annotation.RestController;

@RestController

public class HomeController {

    @GetMapping("/")

    public String welcome() {

        return "Welcome";

    }

    @GetMapping("/admin")

    public String welcomeAdmin() {

        return "Welcome Admin";

    }

}

```

=====

react only 2 passed:

```

import React, { createContext, useState } from 'react';

export const FilterContext = createContext();

export const FilterProvider = ({ children, value }) => {

    if (value) {

        return (

            <FilterContext.Provider value={value}>

```

```

    {children}

  </FilterContext.Provider>

);

}

const [filter, setFilter] = useState('All');

return (

  <FilterContext.Provider value={{ filter, setFilter }}>

    {children}

  </FilterContext.Provider>

);

};

```

```

//low

```

```

import React, { useContext, useMemo } from 'react';

import { ProductContext } from '../contexts/ProductContext';

const LowStockAlert = () => {

  const { products } = useContext(ProductContext);

  // Memoize low stock products for performance

  const lowStockProducts = useMemo(

    () => products.filter(product => product.quantity < 10),

    [products]

  );

  if (lowStockProducts.length === 0) return null;

```

```

return (

  <div style={{ border: '1px solid red', padding: '10px', marginBottom: '10px' }}>

    <h3>Low Stock Alerts</h3>

    <ul>

      {lowStockProducts.map(product => (

        <li key={product.id}>

          {product.name} - Quantity: {product.quantity}

        </li>

      ))}

    </ul>

  </div>

);

};

export default LowStockAlert;

```

```

//productlist

import React, { useContext, useMemo } from "react";

import { ProductContext } from "../contexts/ProductContext";

import { FilterContext } from "../contexts/FilterContext";

import LowStockAlert from "../LowStockAlert";

import ProductFilter from "../ProductFilter";

import '../App.css';

const ProductList = () => {

```

```
const { products, setSelectedProduct } = useContext(ProductContext);

const { filter } = useContext(FilterContext);

const filteredProducts = useMemo(() => {

  if (filter === 'All') return products;

  return products.filter(product => product.category === filter);

}, [products, filter]);

return (

  <div className="product-list-container">

    <h2>Product List</h2>

    <ProductFilter />

    <LowStockAlert />

    <table border="1" cellPadding="5" cellSpacing="0" style={{ width: '100%', marginTop:
'10px' }}>

      <thead>

        <tr>

          <th>Name</th>

          <th>Category</th>

          <th>Price ($)</th>

          <th>Quantity</th>

        </tr>

      </thead>

      <tbody>

        {filteredProducts.map(product => (

          <tr key={product.id}>

            <td>
```

```
      <button onClick={() => setSelectedProduct(product)} style={{ all: 'unset', cursor:
'pointer', color: 'blue', textDecoration: 'underline' }}>
```

```
        {product.name}
```

```
      </button>
```

```
    </td>
```

```
    <td>{product.category}</td>
```

```
    <td>{product.price.toFixed(2)}</td>
```

```
    <td>{product.quantity}</td>
```

```
  </tr>
```

```
  )))
```

```
    <tr>
```

```
      <td colspan="4" style={{ textAlign: 'center' }}>No products found.</td>
```

```
    </tr>
```

```
  })
```

```
</tbody>
```

```
</table>
```

```
</div>
```

```
);
```

```
};
```

```
export default ProductList;
```

```
///productfilter
```



```

import React, { useContext, useEffect, useState } from 'react';

import { FilterContext } from '../contexts/FilterContext';

import { ProductContext } from '../contexts/ProductContext';

const ProductFilter = () => {

  const { filter, setFilter } = useContext(FilterContext);

  const { products } = useContext(ProductContext);

  const [categories, setCategories] = useState([]);

  useEffect(() => {

    // Extract unique categories from products

    const cats = Array.from(new Set(products.map(p => p.category)));

    setCategories(cats);

  }, [products]);

  const handleFilterChange = (e) => {

    setFilter(e.target.value);

  };

  return (

    <div style={{ marginBottom: '10px' }}>

      <label htmlFor="categoryFilter">Filter by Category: </label>

      <select id="categoryFilter" value={filter} onChange={handleFilterChange}>

        <option value="All">All</option>

        {categories.map(cat => (

          <option key={cat} value={cat}>{cat}</option>

        ))}

      </select>

```

```
</div>
```

```
);
```

```
};
```

```
export default ProductFilter;
```

```
//productdetails:
```

```
import React, { useContext, useState, useEffect } from 'react';
```

```
import { ProductContext } from '../contexts/ProductContext';
```

```
import { FilterContext } from '../contexts/FilterContext';
```

```
const ProductDetail = () => {
```

```
  const { selectedProduct, updateProduct, setSelectedProduct, loading } =  
  useContext(ProductContext);
```

```
  const { setFilter } = useContext(FilterContext);
```

```
  const [editedProduct, setEditedProduct] = useState({
```

```
    id: null,
```

```
    name: "",
```

```
    category: "",
```

```
    price: "",
```

```
    quantity: ""
```

```
  });
```

```
  const [errors, setErrors] = useState({});
```

```
  useEffect(() => {
```

```
    if (selectedProduct) {
```

```

setEditedProduct({
  id: selectedProduct.id || null,
  name: selectedProduct.name || "",
  category: selectedProduct.category || "",
  price: selectedProduct.price !== undefined ? String(selectedProduct.price) : "",
  quantity: selectedProduct.quantity !== undefined ? String(selectedProduct.quantity) : ""
});
} else {
  setEditedProduct({
    id: null,
    name: "",
    category: "",
    price: "",
    quantity: ""
  });
}

setErrors({});
}, [selectedProduct]);

const validate = () => {
  const errs = {};

  if (!editedProduct.name.trim()) errs.name = 'Name is required';

  if (!editedProduct.category.trim()) errs.category = 'Category is required';

  if (editedProduct.price === "" || isNaN(editedProduct.price) || Number(editedProduct.price) <
0)

```

```

    errs.price = 'Price must be a non-negative number';

    if (editedProduct.quantity === '' || !Number.isInteger(Number(editedProduct.quantity)) ||
    Number(editedProduct.quantity) < 0)

    errs.quantity = 'Quantity must be a non-negative integer';

    setErrors(errs);

    return Object.keys(errs).length === 0;

};

const handleChange = (e) => {

    const { name, value } = e.target;

    setEditedProduct(prev => ({

        ...prev,

        [name]: value

    }));

};

const handleSave = () => {

    if (!validate()) return;

    const productToSave = {

        ...editedProduct,

        price: Number(editedProduct.price),

        quantity: Number(editedProduct.quantity),

        id: editedProduct.id || Date.now()

    };

    updateProduct(productToSave).then(() => {

        setSelectedProduct(productToSave);

        setFilter('All');

```

```
});  
  
};  
  
return (  
  
  <div className="product-detail-container">  
  
    <h2>{editedProduct.id ? 'Edit Product' : 'Add New Product'}</h2>  
  
    <form onSubmit={e => e.preventDefault()}>  
  
      <div>  
  
        <label>Name:</label><br />  
  
        <input  
  
          name="name"  
  
          value={editedProduct.name}  
  
          onChange={handleChange}  
  
          disabled={loading}  
  
        />  
  
        {errors.name && <div style={{ color: 'red' }}>{errors.name}</div>}  
  
      </div>  
  
      <div>  
  
        <label>Category:</label><br />  
  
        <input  
  
          name="category"  
  
          value={editedProduct.category}  
  
          onChange={handleChange}  
  
          disabled={loading}  
  
        />  

```

```
{errors.category && <div style={{ color: 'red' }}>{errors.category}</div>}
```

```
</div>
```

```
<div>
```

```
<label>Price:</label><br />
```

```
<input
```

```
  name="price"
```

```
  type="number"
```

```
  step="0.01"
```

```
  value={editedProduct.price}
```

```
  onChange={handleChange}
```

```
  disabled={loading}
```

```
{errors.price && <div style={{ color: 'red' }}>{errors.price}</div>}
```

```
</div>
```

```
<div>
```

```
<label>Quantity:</label><br />
```

```
<input
```

```
  name="quantity"
```

```
  type="number"
```

```
  value={editedProduct.quantity}
```

```
  onChange={handleChange}
```

```
  disabled={loading}
```

```
{errors.quantity && <div style={{ color: 'red' }}>{errors.quantity}</div>}
```

```

    </div>

    <button onClick={handleSave} disabled={loading}>

      {loading ? 'Saving...' : 'Save'}

    </button>

  </form>

</div>

);

};

export default ProductDetail;

//productcontext:

import React, { createContext, useState, useEffect } from 'react';

import productsData from '../data/products';

export const ProductContext = createContext();

export const ProductProvider = ({ children, value }) => {

  // Use overridden context value for testing if provided

  if (value) {

    return (

      <ProductContext.Provider value={value}>

        {children}

      </ProductContext.Provider>

    );

  }

```

```

}

const [products, setProducts] = useState(productsData);

const [selectedProduct, setSelectedProduct] = useState(null);

const [loading, setLoading] = useState(false);

// Remove async fetch for test stability

const updateProduct = (updatedProduct) => {

  setLoading(true);

  return new Promise(resolve => {

    setTimeout(() => {

      // Update products array and also mutate imported products array for test

      const index = products.findIndex(p => p.id === updatedProduct.id);

      let newProducts;

      if (index !== -1) {

        newProducts = [...products];

        newProducts[index] = updatedProduct;

        products[index] = updatedProduct; // mutate imported array

      } else {

        newProducts = [...products, updatedProduct];

        products.push(updatedProduct); // mutate imported array

      }

      setProducts(newProducts);

      setLoading(false);

      resolve(true);

    }, 100); // shorter delay for test speed

```



```

});

};

return (

<ProductContext.Provider

value={{

products,

selectedProduct,

setSelectedProduct,

updateProduct,

loading

}}

>

{children}

</ProductContext.Provider>

);

};

```

MISCELLANEOUS

springboot practice

1. JDBC

```
package com.student.api.controller;
```

```
import com.student.api.domain.Student;
```

```
import org.springframework.jdbc.core.RowMapper;
```

```
import java.sql.ResultSet;
```

```
import java.sql.SQLException;
```

```
public class StudentRowMapper implements RowMapper<Student> {
```

```
    /**
```

```
    *
```

```
    * @param rs Database ResultSet object. Get database data with the column names "ID"
    and "NAME". Remember, "ID" column is Long data type and "NAME" column is String data
    type.
```

```
    * @param rowNum If you get data with column names as described above, you don't need
    to use rowNum parameter
```

```
    * @return Student object with the mapped values from database
```

```
    */
```

```
@Override
```

```
public Student mapRow(ResultSet rs, int rowNum) throws SQLException {
```

```
    Long id = rs.getLong("ID");
```

```
    String name = rs.getString("NAME");
```

```
    return new Student(id, name);
```

```
}
```

```
}
```

```
package com.student.api.controller;
```

```
import com.student.api.domain.Student;
```

```
import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.dao.EmptyResultDataAccessException;
```

```
import org.springframework.http.ResponseEntity;
```

```
import org.springframework.jdbc.core.namedparam.MapSqlParameterSource;
```

```
import org.springframework.jdbc.core.namedparam.NamedParameterJdbcTemplate;
```

```
import org.springframework.web.bind.annotation.*;
```

```
import java.util.List;
```

```
/**
```

```
 * REST controller for managing student system process. Use {@link StudentRowMapper} to  
map database rows to Student entity object.
```

```
 */
```

```
@RestController
```

```
@RequestMapping("/api/v1")
```

```
public class StudentController {
```

```
    // use JdbcTemplate to query for students against database
```

```
    @Autowired
```

```
    private final NamedParameterJdbcTemplate jdbcTemplate;
```

```
    public StudentController(NamedParameterJdbcTemplate jdbcTemplate) {
```

```
this.jdbcTemplate = jdbcTemplate;

}
```

```
/**
```

```
 * {@code GET /students} : get all the Students.
```

```
 *
```

```
 * @return the {@link ResponseEntity} with status {@code 200 (OK)} and the list
 * of students in body.
```

```
 */
```

```
@GetMapping("/students")
```

```
public ResponseEntity<List<Student>> getAllStudents() {
```

```
    String sql = "SELECT * FROM student";
```

```
    List<Student> students = jdbcTemplate.query(sql, new StudentRowMapper());
```

```
    return ResponseEntity.ok(students);
```

```
}
```

```
/**
```

```
 * {@code GET /students/{id}} : get the "id" Student.
```

```
 *
```

```
 * @param id the id of the student to retrieve.
```

```
 * @return the {@link ResponseEntity} with status {@code 200 (OK)} and with body
```

```
 * the student, or if does not exist, return with status "noContent".
```

```
 */
```

```
@GetMapping("/students/{id}")
```

```

public ResponseEntity<Student> getStudent(@PathVariable Long id) {

    try{

        String sql = "SELECT * FROM student WHERE id = :id";

        Student student = jdbcTemplate.queryForObject(sql, new MapSqlParameterSource("id",
id), new StudentRowMapper());

        return ResponseEntity.ok(student);

    }

    catch(EmptyResultDataAccessException ex){

        return ResponseEntity.noContent().build();

    }

}

```

/**

* {@code POST /student} : Create a new student.

*

* @param student the student to create.

* @return the {@link ResponseEntity} with status {@code 200 (OK)} and with

* body the new student

*/

@PostMapping("/students")

```

public ResponseEntity<Void> createStudent(@RequestBody Student student) {

```

```

    String sql = "INSERT INTO student (id, name) VALUES (:id, :name)";

```

```

    int rowsAffected = jdbcTemplate.update(sql,

```

```

        new MapSqlParameterSource()

        .addValue("id", student.getId())

        .addValue("name", student.getName()));

    if(rowsAffected > 0){

        return ResponseEntity.ok().build();

    }else{

        return ResponseEntity.noContent().build();

    }

}

}

/**
 * {@code PUT /student} : Updates an existing student.
 *
 * @param student the student to update.
 * @return the {@link ResponseEntity} with status {@code 200 (OK)} and with body
 * the updated student.
 */

@PutMapping("/students")

public ResponseEntity<Void> updateStudent(@RequestBody Student student) {

    String sql = "UPDATE student SET name = :name WHERE id = :id";

    int rowsAffected = jdbcTemplate.update(sql,

    new MapSqlParameterSource()

    .addValue("id", student.getId())

    .addValue("name", student.getName()));

```

```

        if(rowsAffected > 0){

            return ResponseEntity.ok().build();

        }else{

            return ResponseEntity.noContent().build();

        }

    }

}

/**
 * {@code DELETE /student/:id} : delete the "id" student.
 *
 * @param id the id of the student to delete.
 * @return the {@link ResponseEntity} with status {@code 200 (OK)}.
 */
@DeleteMapping("/students/{id}")
public ResponseEntity<Void> deleteStudent(@PathVariable Long id) {

    String sql = "DELETE FROM student WHERE id = :id";

    int rowsAffected = jdbcTemplate.update(sql,

        new MapSqlParameterSource("id", id));

    if(rowsAffected > 0){

        return ResponseEntity.ok().build();

    }else{

        return ResponseEntity.noContent().build();

    }

}
}

```

}

2.HTML BANK [ONline bank account]

html all testcases pass

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Online Banking: Account Transactions Viewer</title>
```

```
<style>
```

```
body {
```

```
background-color: #f0f0f0;
```

```
}
```

```
form {
```

```
display: flex;
```

```
flex-direction: column;
```

```
width: 50%;
```

```
justify-content: center;
```

```
align-items: center;
```

```
border: 1px solid #fff;
```

```
margin: 0 auto;
```

```
padding: 10px;
```



```
}
```

```
div {
```

```
width: 50%;
```

```
display: flex;
```

```
justify-content: center;
```

```
margin: 4rem auto;
```

```
}
```

```
label {
```

```
width: 20%;
```

```
font-size: 1.2rem;
```

```
}
```

```
select {
```

```
width: 20%;
```

```
}
```

```
table {
```

```
font-family: arial, sans-serif;
```

```
border-collapse: collapse;
```

```
width: 100%;
```

```
}
```

```
td,
```

```
th {
```

```
border: 1px solid #dddddd;
```

```
text-align: left;
```

```
padding: 8px;
```

```
}
```

```
tr.deposit {
```

```
background-color: #d4edda;
```

```
color: #155724;
```

```
}
```

```
tr.withdrawl {
```

```
background-color: #f8d7da;
```

```
color: #721c24;
```

```
}
```

```
a:hover {
```

```
color: orange;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Online Banking: Account Transactions Viewer</h2>
```

```
<div>
```

```
<label for="type">Transaction Type</label>
```

```
<select id="type">
```

```
<option value="">All</option>
```

```
<option value="DEPOSIT">DEPOSIT</option>
```

```
<option value="WITHDRAWL">WITHDRAWL</option>
```

```
</select>
```

```
<button id="search-btn">Search</button>
```

```
</div>
```

```
<div>
```

```
<table>
```

```
<thead>
```

```
<tr>
```

```
<th>Description</th>
```

```
<th>Amount</th>
```

```
<th>Type</th>
```

```
</tr>
```

```
</thead>
```

```
<tbody id="transactionTableBody"></tbody>
```

```
</table>
```

```
</div>
```

```
<script type="text/javascript">
```

```
// Do not change these hardcoded transactions
```

```
const transactions = [
```

```
{
```

```
  description: "Transfer to Mr A",
```

```
  amount: 1000,
```

```
  type: "WITHDRAWAL",
```

```
},
```

```
{
```

```
  description: "Salary March 2022",
```

```
  amount: 50000,
```

```

    type: "DEPOSIT",

  },

  {

    description: "House Rent",

    amount: 4000,

    type: "WITHDRAWAL",

  },

  {

    description: "Receive from Mr B",

    amount: 2000,

    type: "DEPOSIT",

  },

];

const transactionTableBody = document.getElementById("transactionTableBody");

const searchBtn = document.getElementById("search-btn");

const dropdown = document.getElementById("type");

// Populate transactions based on selected type

searchBtn.addEventListener("click", (e) => {

  e.preventDefault();

  const selectedType = dropdown.value;

  populateTransactions(selectedType);

});

function populateTransactions(selectedType = "") {

  transactionTableBody.innerHTML = "";

```

```
const filteredTransactions = getTransactions(selectedType);

filteredTransactions.forEach((transaction) => {

  const row = document.createElement("tr");

  row.className = transaction.type.toLowerCase();

  row.innerHTML = `

    <td>${transaction.description}</td>

    <td>${transaction.amount}</td>

    <td>${transaction.type}</td>

  `;

  transactionTableBody.appendChild(row);

});

}

function getTransactions(selectedType) {

  if (selectedType === "") {

    return transactions;

  }

  return transactions.filter((transaction) => transaction.type === selectedType);

}

// Populate all transactions initially

populateTransactions();

</script>

</body>

</html>
```

3.REACT PATIENT

PatientInformation.js :

```
import React, { useState, useEffect } from 'react';

import { getPatients } from './PatientService';

import './App.css';

export const PatientInformation = ({ patientID }) => {

  const [patient, setPatient] = useState(null);

  useEffect(() => {

    const fetchPatient = async () => {

      const patients = await getPatients();

      const found = patients.find(p => p.patientID === patientID);

      setPatient(found || null);

    };

    if (patientID) {

      fetchPatient();

    }

  }, [patientID]);

  return (

    <div className="patient-info-container">

      {patient ? (

        <div className="patient-card">

          <h3>Patient Details</h3>
```

<p>Patient ID: {patient.patientID}</p>

<p>Name: {patient.name}</p>

<p>Age: {patient.age}</p>

<p>Gender: {patient.gender}</p>

<p>Condition: {patient.condition}</p>

<p>Last Visit: {patient.lastVisit}</p>

</div>

) : (

<p>No patient found for ID: {patientID}</p>

}}

</div>

);

};

PatientRegistrationForm.js :

```
import React, { useState } from 'react';
```

```
import { addPatient } from './PatientService';
```

```
import './App.css';
```

```
const PatientRegistrationForm = ({ onRegister }) => {
```

```
  const [errors, setErrors] = useState({});
```

```
  const [formData, setFormData] = useState({
```

```
    name: "",
```

```
    age: "",
```

```
    gender: "",
```

```
    condition: "",
```

```
lastVisit: ",

});

const handleChange = (e) => {

const { name, value } = e.target;

setFormData({ ...formData, [name]: value });

};

const isValidDate = (dateString) => {

const regex = /^\\d{4}-\\d{2}-\\d{2}$/;

return regex.test(dateString);

};

const validateForm = () => {

const errs = {};

if (!formData.name.trim()) errs.name = 'Name is required';

if (!formData.age) errs.age = 'Age is required';

else if (isNaN(formData.age) || formData.age <= 0) errs.age = 'Age must be a positive number';

if (!formData.gender) errs.gender = 'Gender is required';

if (!formData.condition.trim()) errs.condition = 'Condition is required';

if (!formData.lastVisit.trim()) errs.lastVisit = 'Last Visit is required';

else if (!isValidDate(formData.lastVisit)) errs.lastVisit = 'Invalid date format (YYYY-MM-DD)';

setErrors(errs);

return Object.keys(errs).length === 0;

};

const handleSubmit = async (e) => {

e.preventDefault();
```



```
if (!validateForm()) return;

const newPatient = {

...formData,

patientID: `P${Date.now().toString().slice(-4)}`

};

await addPatient(newPatient);

if (onRegister) {

onRegister(formData); // matches test expectation

}

setFormData({ name: "", age: "", gender: "", condition: "", lastVisit: "" });

setErrors({});

};

return (

<form className="patient-form" onSubmit={handleSubmit}>

<h3>Register New Patient</h3>

<input name="name" placeholder="Name" value={formData.name}
onChange={handleChange} />

{errors.name && <div className="error">{errors.name}</div>}

<input name="age" placeholder="Age" value={formData.age} onChange={handleChange}
/>

{errors.age && <div className="error">{errors.age}</div>}

<select name="gender" value={formData.gender} onChange={handleChange}>

<option value="">Select Gender</option>

<option>Male</option>

<option>Female</option>
```

```
<option>Other</option>
```

```
</select>
```

```
{errors.gender && <div className="error">{errors.gender}</div>}
```

```
<input name="condition" placeholder="Condition" value={formData.condition}  
onChange={handleChange} />
```

```
{errors.condition && <div className="error">{errors.condition}</div>}
```

```
<input name="lastVisit" placeholder="Last Visit (YYYY-MM-DD)" value={formData.lastVisit}  
onChange={handleChange} />
```

```
{errors.lastVisit && <div className="error">{errors.lastVisit}</div>}
```

```
<button type="submit">Register Patient</button>
```

```
</form>
```

```
);
```

```
};
```

```
export default PatientRegistrationForm;
```

```
PatientService.js :
```

```
import environment from "../environments/environment.ts"
```

```
const API_URL = environment.apiUrl;
```

```
export const getPatients = async () => {
```

```
  const response = await fetch(`${API_URL}/patients`);
```

```
  if (!response.ok) throw new Error("Failed to fetch patients");
```

```
  return await response.json();
```

```
};
```

```
export const addPatient = async (newPatient) => {
```

```
  const response = await fetch(`${API_URL}/patients`, {
```

```
    method: 'POST',
```

```
headers: {  
  
  'Content-Type': 'application/json'  
  
},  
  
body: JSON.stringify(newPatient)  
  
});  
  
if (!response.ok) throw new Error("Failed to add patient");  
  
return await response.json();  
  
};
```

4.PROPERTY REACT

AddProperty.js:

```
import React, { useState } from 'react';  
  
import PropertyService from './PropertyService';  
  
import './App.css';  
  
const AddProperty = () => {  
  
  const [property, setProperty] = useState({  
  
    _id: "",  
  
    type: "",  
  
    location: "",  
  
    price: "",  
  
    rooms: "",  
  
    size: ""  
  
  });
```

```
const [error, setError] = useState(null);

const [message, setMessage] = useState("");

const handleChange = (e) => {

  setProperty({ ...property, [e.target.name]: e.target.value });

};

const handleSubmit = async (e) => {

  e.preventDefault();

  setError(null);

  try {

    await PropertyService.addProperty(property);

    setMessage('Property added successfully!');

  } catch (err) {

    setError(err.message);

  }

};

return (

  <div className="add-property-container">

    <h2>Add New Property</h2>

    {error && <p className="error">{error}</p>}

    {message && <p className="success">{message}</p>}

    <form onSubmit={handleSubmit}>

      {[ 'type', 'location', 'price', 'rooms', 'size', '_id' ].map((field) => (

        <div key={field}>

          <label>{field}</label>
```

```

<input
  name={field}
  value={property[field]}
  onChange={handleChange}
  required
/>

</div>

  )}

<button type="submit">Add Property</button>

</form>

</div>

);

};

```

```
export default AddProperty;
```

PropertyList.js :

```

import React, { useState, useEffect } from 'react';
import PropertyService from './PropertyService';
import { Link } from 'react-router-dom';
import './App.css';

const PropertyList = () => {

  const [properties, setProperties] = useState([]);

  const [loading, setLoading] = useState(true);

  const [error, setError] = useState(null);

  useEffect(() => {

```

```

const fetchProperties = async () => {

  try {

    const data = await PropertyService.getAllProperties();

    setProperties(data);

    setLoading(false);

  } catch (err) {

    setError(err.message);

    setLoading(false);

  }

};

fetchProperties();

}, []);

if (loading) return <p>Loading...</p>;

if (error) return <p>Error: {error}</p>;

return (

<div className="property-list-container">

<h2 className="property-list-header">Properties List</h2>

<ul className="property-list">

  {properties.map((property) => (

<li key={property._id}>

<Link to={` /properties/${property._id}`}>

  {property.location} - {property.type}

</Link>

</li>

```

```
    )}
```

```
</ul>
```

```
</div>
```

```
);
```

```
};
```

```
export default PropertyList;
```

PropertyDetail.js :

```
import React, { useState, useEffect } from 'react';
```

```
import { useParams } from 'react-router-dom';
```

```
import PropertyService from './PropertyService';
```

```
import './App.css';
```

```
const PropertyDetail = () => {
```

```
  const { propertyID } = useParams();
```

```
  const [property, setProperty] = useState(null);
```

```
  const [loading, setLoading] = useState(true);
```

```
  const [error, setError] = useState(null);
```

```
  useEffect(() => {
```

```
    const fetchProperty = async () => {
```

```
      try {
```

```
        const data = await PropertyService.getPropertyByID(propertyID);
```

```
        setProperty(data[0]); // use first element
```

```
        setLoading(false);
```

```
      } catch (err) {
```

```
        setError(err.message);
```

```

    setLoading(false);

  }

};

fetchProperty();

}, [propertyID]);

if (loading) return <p>Loading...</p>;

if (error) return <p>Error: {error}</p>;

return (

<div className="property-detail-container">

<h2>Property Details</h2>

<p>Type: {property.type}</p>

<p>Location: {property.location}</p>

<p>Price: {property.price}</p>

<p>Rooms: {property.rooms}</p>

<p>Size: {property.size}</p>

</div>

);

};

export default PropertyDetail;

```

PropertyService.js :

```

const API_URL = `http://localhost:3000/properties`;

const PropertyService = {

  getAllProperties: async () => {

    const response = await fetch(API_URL);

```



```
if (!response.ok) {

throw new Error('Failed to fetch properties');

}

return response.json();

},

getPropertyByID: async (propertyID) => {

const response = await fetch(`${API_URL}?_id=${propertyID}`);

if (!response.ok) {

throw new Error('Failed to fetch property details');

}

return response.json(); // returns an array

},

addProperty: async (newProperty) => {

const response = await fetch(API_URL, {

method: 'POST',

headers: { 'Content-Type': 'application/json' },

body: JSON.stringify(newProperty),

});

if (!response.ok) {

throw new Error('Failed to add property');

}

return response.json();

},

};
```

```
export default PropertyService;
```

5.

REACT QUESTION

Q. Build a Sales Dashboard Application

Dashboard.jsx

```
import axios from "axios";
```

```
import React, { useEffect , useState} from "react";
```

```
import "./Dashboard.css";
```

```
import { calculateTotalSales, calculateTotalCashSale, calculateTotalCreditSale,  
calculateBuyerWithMostSale} from './Reports';
```

```
function Dashboard(){
```

```
  const App=()=>{
```

```
    const[data, setData]=useState([]);
```

```
    useEffect(()=>{
```

```
      (async)=>{
```

```
        const result=await axios.get('/sales.json');
```

```
        setData(result.data);
```

```
      })();
```

```
    },[])
```

```
    return (
```

```
      <div className="dashboard">
```

```
        <div className="card">
```

```
          <h2>Total Sales</h2>
```

```

    <p>{calculateTotalSales(data)}</p>

</div>

<div className="card">

  <h2>Total Cash Sales</h2>

  <p>{calculateTotalCashSale(data)}</p>

</div>

<div className="card">

  <h2>Total Credit Sales</h2>

  <p>{calculateTotalCreditSale(data)}</p>

</div>

<div className="card">

  <h2>Buyer with Most Sales</h2>

  <p>{calculateBuyerWithMostSale(data).buyerName}</p>

  <p>{calculateBuyerWithMostSale(data).saleTotal}</p>

</div>

</div>

);

}

}

```

```
export default Dashboard;
```

Reports.js

```
import axios from "axios";
```

```
export const getSalesData = async () => {
```

```
let { data } = await axios.get(`/sales.json`);

return data;

};

export const calculateTotalSales = (sales) => {

  return sales.reduce((total, sale) => total + sale.saleTotal, 0);

};

export const calculateTotalCashSale = (sales) => {

  return sales.filter(sale => sale.creditCard === false)

    .reduce((total, sale) => total + sale.saleTotal, 0);

};

export const calculateTotalCreditSale = (sales) => {

  return sales.filter(sale => sale.creditCard === true)

    .reduce((total, sale) => total + sale.saleTotal, 0);

};

export const calculateBuyerWithMostSale = (sales) => {

  const buyerMap = {};

  for (const sale of sales) {

    if (!buyerMap[sale.buyerName]) {

      buyerMap[sale.buyerName] = 0;

    }

    buyerMap[sale.buyerName] += sale.saleTotal;

  }

  let maxBuyer = null;

  let maxTotal = 0;
```

```
for(const[buyer,total] of Object.entries(buyerMap)){  
  if(total>maxTotal){  
    maxBuyer=buyer;  
    maxTotal=total;  
  }  
}  
  
return {  
  buyerName:maxBuyer,  
  saleTotal:maxTotal  
};  
};
```

6.HTML/CSS/JS Question

Q. Bank Management System Form

index.html

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<title>Transaction Filter</title>
```

```
<style>
```

```
body {
```

```
  background-color: gray;
```

```
}
```

```
</style>

</head>

<body>

  <div>

    <select id="transactionType">

      <option value="all">All</option>

      <option value="deposit">Deposits</option>

      <option value="withdrawal">Withdrawals</option>

    </select>

    <ul id="transactionList"></ul>

  </div>

</body>

</html>
```

index.css

```
body {

  background-color: gray;

}
```

index.js

```
const transactions = [

  { type: "deposit", amount: 100 },

  { type: "withdrawal", amount: 50 },

  { type: "deposit", amount: 200 },
```

```

    { type: "withdrawal", amount: 30 },

    { type: "deposit", amount: 150 }

];

function filterTransactions(type, container) {

    container.innerHTML = ""; // Clear previous entries

    const filtered = type === "all"

    ? transactions

    : transactions.filter(txn => txn.type === type);

    filtered.forEach(txn => {

        const li = document.createElement("li");

        li.textContent = `${txn.type.toUpperCase()}: ${txn.amount}`;

        container.appendChild(li);

    });

}

module.exports = filterTransactions;

```

html practice

```

<!DOCTYPE html>

<html>

<head>

<title>Online Banking: Account Transactions Viewer</title>

<style>

```

```
body {  
  
  background-color: #f0f0f0;  
  
}
```

```
form {  
  
  display: flex;  
  
  flex-direction: column;  
  
  width: 50%;  
  
  justify-content: center;  
  
  align-items: center;  
  
  border: 1px solid #fff;  
  
  margin: 0 auto;  
  
  padding: 10px;  
  
}
```

```
div {  
  
  width: 50%;  
  
  display: flex;  
  
  justify-content: center;  
  
  margin: 4rem auto;  
  
}
```

```
label {  
  
  width: 20%;
```



```
font-size: 1.2rem;  
  
}
```

```
select {  
  
width: 20%;  
  
}
```

```
table {  
  
font-family: arial, sans-serif;  
  
border-collapse: collapse;  
  
width: 100%;  
  
}
```

```
td,  
  
th {  
  
border: 1px solid #dddddd;  
  
text-align: left;  
  
padding: 8px;  
  
}
```

```
tr.deposit {  
  
background-color: #d4edda;  
  
color: #155724;  
  
}
```

```
tr.withdrawl {
```

```
    background-color: #f8d7da;
```

```
    color: #721c24;
```

```
}
```

```
a:hover {
```

```
    color: orange;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>Online Banking: Account Transactions Viewer</h2>
```

```
<div>
```

```
<label for="type">Transaction Type</label>
```

```
<select id="type">
```

```
<option value="">All</option>
```

```
<option value="DEPOSIT">DEPOSIT</option>
```

```
<option value="WITHDRAWAL">WITHDRAWAL</option>
```

```
</select>
```

```
<button id="search-btn">Search</button>
```

```
</div>
```

```
<div>

<table>

<thead>

<tr>

<th>Description</th>

<th>Amount</th>

<th>Type</th>

</tr>

</thead>

<tbody id="transactionTableBody"></tbody>

</table>

</div>
```

```
<script type="text/javascript">

// Do not change these hardcoded transactions

const transactions = [

{

description: "Transfer to Mr A",

amount: 1000,

type: "WITHDRAWAL",

},

{

description: "Salary March 2022",

amount: 50000,
```

```
    type: "DEPOSIT",  
  },  
  {  
    description: "House Rent",  
    amount: 4000,  
    type: "WITHDRAWAL",  
  },  
  {  
    description: "Receive from Mr B",  
    amount: 2000,  
    type: "DEPOSIT",  
  },  
];
```

```
const transactionTableBody = document.getElementById("transactionTableBody");  
  
const searchBtn = document.getElementById("search-btn");  
  
const dropdown = document.getElementById("type");  
  
// Populate transactions based on selected type  
  
searchBtn.addEventListener("click", (e) => {  
  e.preventDefault();  
  
  const selectedType = dropdown.value;  
  
  populateTransactions(selectedType);  
});
```

```

function populateTransactions(selectedType = "") {

  transactionTableBody.innerHTML = "";

  const filteredTransactions = getTransactions(selectedType);

  filteredTransactions.forEach((transaction) => {

    const row = document.createElement("tr");

    row.className = transaction.type.toLowerCase();

    row.innerHTML = `

      <td>${transaction.description}</td>

      <td>${transaction.amount}</td>

      <td>${transaction.type}</td>

    `;

    transactionTableBody.appendChild(row);

  });

}

function getTransactions(selectedType) {

  if (selectedType === "") {

    return transactions;

  }

  return transactions.filter((transaction) => transaction.type === selectedType);

```

```
}
```

```
// Populate all transactions initially
```

```
populateTransactions();
```

```
</script>
```

```
</body>
```

```
</html>
```

```
=====
```

REACT CODE:

AddProperty.js:

```
import React, { useState } from 'react';
```

```
import PropertyService from './PropertyService';
```

```
import './App.css';
```

```
const AddProperty = () => {
```

```
  const [property, setProperty] = useState({
```

```
    _id: "",
```

```
    type: "",
```

```
    location: "",
```

```
price: ",  
  
rooms: ",  
  
size: "  
  
});  
  
const [error, setError] = useState(null);  
  
const [message, setMessage] = useState("");  
  
const handleChange = (e) => {  
  
  setProperty({ ...property, [e.target.name]: e.target.value });  
  
};  
  
const handleSubmit = async (e) => {  
  
  e.preventDefault();  
  
  setError(null);  
  
  try {  
  
    await PropertyService.addProperty(property);  
  
    setMessage('Property added successfully!');  
  
  } catch (err) {  
  
    setError(err.message);  
  
  }  
  
};  
  
return (  
  
<div className="add-property-container">  
  
<h2>Add New Property</h2>  
  
{error && <p className="error">{error}</p>}  
  
{message && <p className="success">{message}</p>}
```

```

<form onSubmit={handleSubmit}>

  {[ 'type', 'location', 'price', 'rooms', 'size', '_id'].map((field) => (

    <div key={field}>

      <label>{field}</label>

      <input

        name={field}

        value={property[field]}

        onChange={handleChange}

        required

      />

    </div>

  )]}

  <button type="submit">Add Property</button>

</form>

</div>

);

};

export default AddProperty;

```

PropertyList.js :

```

import React, { useState, useEffect } from 'react';

import PropertyService from './PropertyService';

import { Link } from 'react-router-dom';

import './App.css';

const PropertyList = () => {

```



```
const [properties, setProperties] = useState([]);

const [loading, setLoading] = useState(true);

const [error, setError] = useState(null);

useEffect(() => {

  const fetchProperties = async () => {

    try {

      const data = await PropertyService.getAllProperties();

      setProperties(data);

      setLoading(false);

    } catch (err) {

      setError(err.message);

      setLoading(false);

    }

  };

  fetchProperties();

}, []);

if (loading) return <p>Loading...</p>;

if (error) return <p>Error: {error}</p>;

return (

  <div className="property-list-container">

    <h2 className="property-list-header">Properties List</h2>

    <ul className="property-list">

      {properties.map((property) => (

        <li key={property._id}>
```

```
<Link to={` /properties/${property._id}`}>
```

```
  {property.location} - {property.type}
```

```
</Link>
```

```
</li>
```

```
  )}}
```

```
</ul>
```

```
</div>
```

```
);
```

```
};
```

```
export default PropertyList;
```

```
PropertyDetail.js :
```

```
import React, { useState, useEffect } from 'react';
```

```
import { useParams } from 'react-router-dom';
```

```
import PropertyService from './PropertyService';
```

```
import './App.css';
```

```
const PropertyDetail = () => {
```

```
  const { propertyID } = useParams();
```

```
  const [property, setProperty] = useState(null);
```

```
  const [loading, setLoading] = useState(true);
```

```
  const [error, setError] = useState(null);
```

```
  useEffect(() => {
```

```
    const fetchProperty = async () => {
```

```
      try {
```

```
        const data = await PropertyService.getPropertyByID(propertyID);
```

```
setProperty(data[0]); // use first element

setLoading(false);

} catch (err) {

setError(err.message);

setLoading(false);

}

};

fetchProperty();

}, [propertyID]));

if (loading) return <p>Loading...</p>;

if (error) return <p>Error: {error}</p>;

return (

<div className="property-detail-container">

<h2>Property Details</h2>

<p>Type: {property.type}</p>

<p>Location: {property.location}</p>

<p>Price: {property.price}</p>

<p>Rooms: {property.rooms}</p>

<p>Size: {property.size}</p>

</div>

);

};

export default PropertyDetail;
```

PropertyService.js :

```
const API_URL = `http://localhost:3000/properties`;

const PropertyService = {

  getAllProperties: async () => {

    const response = await fetch(API_URL);

    if (!response.ok) {

      throw new Error('Failed to fetch properties');

    }

    return response.json();

  },

  getPropertyByID: async (propertyID) => {

    const response = await fetch(`${API_URL}?_id=${propertyID}`);

    if (!response.ok) {

      throw new Error('Failed to fetch property details');

    }

    return response.json(); // returns an array

  },

  addProperty: async (newProperty) => {

    const response = await fetch(API_URL, {

      method: 'POST',

      headers: { 'Content-Type': 'application/json' },

      body: JSON.stringify(newProperty),

    });

    if (!response.ok) {

      throw new Error('Failed to add property');
```

```

}

return response.json();

},

};

export default PropertyService;

```

PatientInformation.js :

```

import React, { useState, useEffect } from 'react';

import { getPatients } from './PatientService';

import './App.css';

export const PatientInformation = ({ patientID }) => {

  const [patient, setPatient] = useState(null);

  useEffect(() => {

    const fetchPatient = async () => {

      const patients = await getPatients();

      const found = patients.find(p => p.patientID === patientID);

      setPatient(found || null);

    };

    if (patientID) {

      fetchPatient();

    }

  }, [patientID]);

  return (

    <div className="patient-info-container">

```

```

{patient ? (

<div className="patient-card">

<h3>Patient Details</h3>

<p>Patient ID: {patient.patientID}</p>

<p>Name: {patient.name}</p>

<p>Age: {patient.age}</p>

<p>Gender: {patient.gender}</p>

<p>Condition: {patient.condition}</p>

<p>Last Visit: {patient.lastVisit}</p>

</div>

) : (

<p>No patient found for ID: {patientID}</p>

)}

</div>

);

};

```

PatientRegistrationForm.js :

```

import React, { useState } from 'react';

import { addPatient } from './PatientService';

import './App.css';

const PatientRegistrationForm = ({ onRegister }) => {

  const [errors, setErrors] = useState({});

  const [formData, setFormData] = useState({

    name: "",

```

```
age: ",
gender: ",
condition: ",
lastVisit: ",
});

const handleChange = (e) => {

const { name, value } = e.target;

setFormData({ ...formData, [name]: value });

};

const isValidDate = (dateString) => {

const regex = /^\\d{4}-\\d{2}-\\d{2}$/;

return regex.test(dateString);

};

const validateForm = () => {

const errs = {};

if (!formData.name.trim()) errs.name = 'Name is required';

if (!formData.age) errs.age = 'Age is required';

else if (isNaN(formData.age) || formData.age <= 0) errs.age = 'Age must be a positive
number';

if (!formData.gender) errs.gender = 'Gender is required';

if (!formData.condition.trim()) errs.condition = 'Condition is required';

if (!formData.lastVisit.trim()) errs.lastVisit = 'Last Visit is required';

else if (!isValidDate(formData.lastVisit)) errs.lastVisit = 'Invalid date format (YYYY-MM-DD)';

setErrors(errs);
```

```
return Object.keys(errs).length === 0;

};

const handleSubmit = async (e) => {

  e.preventDefault();

  if (!validateForm()) return;

  const newPatient = {

    ...formData,

    patientID: `P${Date.now().toString().slice(-4)}`

  };

  await addPatient(newPatient);

  if (onRegister) {

    onRegister(formData); // matches test expectation

  }

  setFormData({ name: "", age: "", gender: "", condition: "", lastVisit: "" });

  setErrors({});

};

return (

  <form className="patient-form" onSubmit={handleSubmit}>

    <h3>Register New Patient</h3>

    <input name="name" placeholder="Name" value={formData.name}

    onChange={handleChange} />

    {errors.name && <div className="error">{errors.name}</div>}

    <input name="age" placeholder="Age" value={formData.age} onChange={handleChange}

    />

    {errors.age && <div className="error">{errors.age}</div>}
```



```

<select name="gender" value={formData.gender} onChange={handleChange}>

<option value="">Select Gender</option>

<option>Male</option>

<option>Female</option>

<option>Other</option>

</select>

{errors.gender && <div className="error">{errors.gender}</div>}

<input name="condition" placeholder="Condition" value={formData.condition}
onChange={handleChange} />

{errors.condition && <div className="error">{errors.condition}</div>}

<input name="lastVisit" placeholder="Last Visit (YYYY-MM-DD)" value={formData.lastVisit}
onChange={handleChange} />

{errors.lastVisit && <div className="error">{errors.lastVisit}</div>}

<button type="submit">Register Patient</button>

</form>

);

};

```

```

export default PatientRegistrationForm;

```

```

PatientService.js :

```

```

import environment from "../environments/environment.ts"

```

```

const API_URL = environment.apiUrl;

```

```

export const getPatients = async () => {

```

```

  const response = await fetch(`${API_URL}/patients`);

```

```

  if (!response.ok) throw new Error("Failed to fetch patients");

```

```

    return await response.json();
  };

  export const addPatient = async (newPatient) => {

    const response = await fetch(`${API_URL}/patients`, {

      method: 'POST',

      headers: {

        'Content-Type': 'application/json'

      },

      body: JSON.stringify(newPatient)

    });

    if (!response.ok) throw new Error("Failed to add patient");

    return await response.json();

  };

```

final exam HTML Css Jss answersss

```

const transactions = [

  { type: "deposit", amount: 100 },

  { type: "withdrawal", amount: 50 },

  { type: "deposit", amount: 200 },

```

```
{ type: "withdrawal", amount: 30 },  
  
{ type: "deposit", amount: 150 }  
  
];
```

```
function getTransactions() {  
  
    const transactionType = document.getElementById("transactionType"); // dropdown  
  
    const transactionList = document.getElementById("transactionList"); // display area  
  
  
    // Call filter when the dropdown changes  
  
    transactionType.addEventListener("change", function () {  
  
        filterTransactions(transactionType.value, transactionList);  
  
    });  
  
  
    // Initial load - show all transactions  
  
    filterTransactions(transactionType.value, transactionList);  
  
}
```

```
// Function to filter and display transactions based on the selected type  
  
function filterTransactions(transactionType, transactionList) {  
  
    transactionList.innerHTML = ""; // Clear previous transactions  
  
  
  
    // Filter the transactions based on selected type  
  
    const filtered = transactionType === "all"  
  
    ? transactions
```

```
: transactions.filter(txn => txn.type === transactionType);
```

```
// Display filtered transactions
```

```
if (filtered.length === 0) {
```

```
  transactionList.innerHTML = "<p>No transactions found.</p>";
```

```
  return;
```

```
}
```

```
filtered.forEach(txn => {
```

```
  const listItem = document.createElement("li");
```

```
  listItem.textContent = `${txn.type.toUpperCase()}: ${txn.amount}`;
```

```
  transactionList.appendChild(listItem);
```

```
});
```

```
}
```

```
// Export for testing or use in other modules
```

```
module.exports = filterTransactions;
```

```
body {
```

```
  background-color: gray;
```

```
  font-family: Arial, sans-serif;
```

```
}
```

```
/* Style deposit transactions in green */
```

```
.deposit {  
  
  color: green;  
  
  font-weight: bold;  
  
}
```

```
/* Style withdrawal transactions in red */
```

```
.withdrawal {  
  
  color: red;  
  
  font-weight: bold;  
  
}
```

```
/* Style links on hover */
```

```
a:hover {  
  
  color: blue;  
  
  text-decoration: underline;  
  
}
```

```
/* Form Inputs Styling */
```

```
input, select, textarea {  
  
  border: 1px solid #999;  
  
  border-radius: 5px;  
  
}
```

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Account Transactions</title>

<!-- Link to external CSS file -->

<link rel="stylesheet" type="text/css" href="./index.css">

</head>

<body>

<h1>Account Transactions</h1>

<label for="transactionType">Select Transaction Type:</label>

<!-- //create select with id transactionType -->

<!-- //create ul wit id transactionList -->

<form action="#" method="post">

<label for="fullName">Full Name:</label>

<input type="text" id="fullName" name="fullName" style="border: 1px solid #ccc;"> <!--
Inline CSS -->

<label for="dob">Date of Birth:</label>

<input type="date" id="dob" name="dob">

<label>Gender:</label>

<input type="radio" id="male" name="gender" value="male">

<label for="male" style="display:inline;">Male</label>

<input type="radio" id="female" name="gender" value="female">
```

<label for="female" style="display:inline;">Female</label>

<label for="address">Address:</label>

<textarea id="address" name="address" rows="4"></textarea>

<label for="accountType">Account Type:</label>

<select id="accountType" name="accountType">

<option value="savings">Savings</option>

<option value="current">Current</option>

<option value="fixed">Fixed Deposit</option>

</select>

<label for="deposit">Initial Deposit:</label>

<input type="number" id="deposit" name="deposit">

<input type="checkbox" id="terms" name="terms">

<label for="terms" style="display:inline;">I agree to the Terms and Conditions</label>

<button type="submit" style="background-color: green; color: white; padding: 10px;">Submit</button>

</form>

<hr>

<h1>Account Transactions</h1>

<label for="transactionType">Select Transaction Type:</label>

<select id="transactionType">

<option value="all">All</option>

<option value="deposit">Deposits</option>

<option value="withdrawal">Withdrawals</option>

```
</select>

<ul id="transactionList"></ul>

</body>

<script src="./index.js" ></script>

</html>
```

Java Programming - 2

```
function solve(statuses) {

    // Write your code here

    ls = statuses.split(";")

    return ls[ls.length -1 ]

}

const statuses = gets();

const result = solve(statuses);

print(result)
```

HTML/CSS(All running) :

index.css :

```
body {  
  
  background-color: gray; /* set background gray */  
  
}  
  
/* Styling for transaction list items */  
  
.deposit {  
  
  color: green; /* set color to green */  
  
}  
  
.withdrawal {  
  
  color: red; /* set color is red */  
  
}  
  
/* Style links to have a custom color when hovered over */  
  
a:hover {  
  
  color: blue;  
  
}
```

index.js :

```
// Sample transactions (type: deposit or withdrawal)  
  
const transactions = [  
  
  { type: "deposit", amount: 100 },  
  
  { type: "withdrawal", amount: 50 },  
  
  { type: "deposit", amount: 200 },  
  
  { type: "withdrawal", amount: 30 },  
  
  { type: "deposit", amount: 150 }
```

```

];

function getTransactions() {

  const transactionType = document.getElementById("transactionType");

  const transactionList = document.getElementById("transactionList");

  const mapLabelToType = {

    all: "all",

    Deposits: "deposit",

    Withdrawals: "withdrawal",

    deposit: "deposit",

    withdrawal: "withdrawal"

  };

  filterTransactions("all", transactionList);

  transactionType.addEventListener("change", () => {

    const selectedType = mapLabelToType[transactionType.value] || "all";

    filterTransactions(selectedType, transactionList);

  });

}

// Function to filter and display transactions

function filterTransactions(type, transactionList) {

  transactionList.innerHTML = ""; // Clear previous list

  const filtered = type === "all" ? transactions : transactions.filter(t => t.type === type);

  filtered.forEach(transaction => {

    const li = document.createElement("li");

    li.textContent = `${transaction.type.toUpperCase()}: ₹${transaction.amount}`;

```

```
li.className = transaction.type;

transactionList.appendChild(li);

});

}

if (typeof document !== "undefined") {

  getTransactions(); // Run only in browser

}

module.exports = filterTransactions;
```

index.html :

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8" />

  <meta name="viewport" content="width=device-width, initial-scale=1.0" />

  <title>Account Transactions</title>

  <!-- Embedded CSS -->

  <style>

    /* Embedded styles to complement external CSS */

    label, select {

      font-weight: bold;

      margin-right: 10px;

    }

    ul#transactionList {

      padding: 10px;
```

```

border: 1px solid #ccc;

width: fit-content;

}

</style>

<!-- External CSS -->

<link rel="stylesheet" href="./index.css" />

</head>

<body>

<h1>Account Transactions</h1>

<label for="transactionType">Select Transaction Type:</label>

<select id="transactionType">

  <option value="all">All</option>

  <option value="deposit">Deposits</option>

  <option value="withdrawal">Withdrawals</option>

</select>

<ul id="transactionList"></ul>

<script src="./index.js"></script>

</body>

</html>

```

SPRING 1 (all testcases passed) :

BookService.java :

```

package com.wecreateproblems.crudcollectionapp.service;

import com.wecreateproblems.crudcollectionapp.entity.Book;

import org.springframework.stereotype.Service;

```

```

import java.util.ArrayList;

import java.util.HashMap;

import java.util.List;

import java.util.Map;

@Service

public class BookService {

    private final Map<Long, Book> books = new HashMap<>();

    public void createBook(Book book) {

        books.put(book.getId(), book); // store book using its id as the key

    }

    public List<Book> getAllBooks() {

        return new ArrayList<>(books.values()); // return all book values as a list

    }

}

```

BookController.java :

```

package com.wecreateproblems.crudcollectionapp.controller;

import com.wecreateproblems.crudcollectionapp.entity.Book;

import com.wecreateproblems.crudcollectionapp.service.BookService;

import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController

@RequestMapping("/books")

public class BookController {

```

```

private final BookService bookService;

@Autowired

public BookController(BookService bookService) {

    this.bookService = bookService;

}

@PostMapping

public void createBook(@RequestBody Book book) {

    bookService.createBook(book); // delegate to service

}

@GetMapping

public List<Book> getAllBooks() {

    return bookService.getAllBooks(); // return list of books

}

}

```

SPRING 2 (all testcases passed) :

SecurityConfig.java :

```

package com.wecp.w3day5task1.config;

import com.wecp.w3day5task1.service.CustomUserDetailsService;

import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.context.annotation.Bean;

import org.springframework.context.annotation.Configuration;

import
org.springframework.security.config.annotation.authentication.builders.AuthenticationManag
erBuilder;

import org.springframework.security.config.annotation.web.builders.HttpSecurity;

```

```

import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;

import
org.springframework.security.config.annotation.web.configuration.WebSecurityConfigurerAd
apter;

import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;

import org.springframework.security.crypto.password.PasswordEncoder;

@Configuration

@EnableWebSecurity

public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Autowired

    private CustomUserDetailsService userDetailsService;

    @Override

    protected void configure(AuthenticationManagerBuilder auth) throws Exception {

        // In-memory authentication with users (USER and ADMIN roles)

        auth.inMemoryAuthentication()

            .withUser("user")

            .password(passwordEncoder().encode("password"))

            .roles("USER")

            .and()

            .withUser("admin")

            .password(passwordEncoder().encode("adminpass"))

            .roles("ADMIN");

    }

    @Override

    protected void configure(HttpSecurity http) throws Exception {

```

http

.httpBasic()

.and()

.authorizeRequests()

.antMatchers("/admin").hasRole("ADMIN") // Only ADMIN role can access /admin

.antMatchers("/").authenticated() // Any authenticated user can access /

.and()

.csrf().disable(); // Disable CSRF for simplicity

}

@Bean

public PasswordEncoder passwordEncoder() {

return new BCryptPasswordEncoder(); // Use BCrypt for password encoding

}

}

HomeController.java :

package com.wecp.w3day5task1.controller;

import org.springframework.web.bind.annotation.GetMapping;

import org.springframework.web.bind.annotation.RestController;

@RestController

public class HomeController {

// Endpoint accessible to all authenticated users

@GetMapping("/")

public String home() {

return "Welcome"; // Return "Welcome" message to any authenticated user


```

    }

    // Endpoint accessible only to users with ADMIN role

    @GetMapping("/admin")

    public String admin() {

        return "Welcome Admin"; // Return "Welcome Admin" message to users with ADMIN role

    }

}

```

CustomUserDetailsService.java :

```

package com.wecp.w3day5task1.service;

import com.wecp.w3day5task1.entity.User;

import com.wecp.w3day5task1.repository.UserRepository;

import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.security.core.authority.SimpleGrantedAuthority;

import org.springframework.security.core.userdetails.UserDetails;

import org.springframework.security.core.userdetails.UsernameNotFoundException;

import org.springframework.stereotype.Service;

import java.util.Collections;

@Service

public class CustomUserDetailsService implements
org.springframework.security.core.userdetails.UserDetailsService {

    @Autowired

    private UserRepository userRepository;

    @Override

    public UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException {

```

```

// Find user by username from the repository

User user = userRepository.findByUsername(username)

    .orElseThrow(() -> new UsernameNotFoundException("User not found"));

// Convert roles to SimpleGrantedAuthority

return new org.springframework.security.core.userdetails.User(

    user.getUsername(),

    user.getPassword(),

    Collections.singleton(new SimpleGrantedAuthority("ROLE_" + user.getRoles()))

);

}

}

```

User.java :

```

package com.wecp.w3day5task1.entity;

import javax.persistence.*;

@Entity

public class User {

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long id;

    private String username;

    private String password;

    private String roles; // "USER" or "ADMIN"

    // Getters and Setters

    public Long getId() {

```

```
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getRoles() {
        return roles;
    }

    public void setRoles(String roles) {
        this.roles = roles;
    }
}
```

UserRepository.java :

```
package com.wecp.w3day5task1.repository;

import com.wecp.w3day5task1.entity.User;

import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.data.jpa.repository.Query;

import java.util.Optional;

public interface UserRepository extends JpaRepository<User, Long> {

    // Manual query for finding a user by username

    @Query("SELECT u FROM User u WHERE u.username = ?1")

    Optional<User> findByUsername(String username);

}
```

REACT :

```
// ProductContext.js

import React, { createContext, useState, useEffect } from 'react';

import initialProducts from '../data/products';

export const ProductContext = createContext();

export const ProductProvider = ({ children }) => {

    const [products, setProducts] = useState(initialProducts);

    const [selectedProduct, setSelectedProduct] = useState(null);
```

```

const updateProduct = (updatedProduct) => {

  setProducts(prev =>

    prev.map(product =>

      product.id === updatedProduct.id ? updatedProduct : product

    )

  );

};

```

```

const addProduct = (newProduct) => {

  newProduct.id = products.length + 1; // simple id assignment

  setProducts([...products, newProduct]);

};

```

```

return (

<ProductContext.Provider

  value={{

    products,

    selectedProduct,

    setSelectedProduct,

    updateProduct,

    addProduct

  }}

>

```

```
    {children}

  </ProductContext.Provider>

);

};
```

```
// ProductList.js
```

```
import React, { useContext } from "react";

import { ProductContext } from "../contexts/ProductContext";

import { FilterContext } from "../contexts/FilterContext";

import LowStockAlert from "../LowStockAlert";

import ProductFilter from "../ProductFilter";

import '../App.css';

const ProductList = () => {

  const { products, setSelectedProduct } = useContext(ProductContext);

  const { filter } = useContext(FilterContext);

  const filteredProducts =

    filter === "All"

    ? products

    : products.filter((p) => p.category === filter);
```

```
return (  
  
  <div className="product-list-container">  
  
    <h2>Product List</h2>  
  
    <ProductFilter />  
  
    <LowStockAlert />  
  
    <div className="product-grid">  
  
      {filteredProducts.map((product) => (  
  
        <button  
  
          key={product.id}  
  
          onClick={() => setSelectedProduct(product)}  
  
        >  
  
          {product.name}  
  
        </button>  
  
      )})  
  
    </div>  
  
  </div>  
  
);  
};
```

```
export default ProductList;
```

```
// LowStockAlert.js
```

```

import React, { useContext } from 'react';

import { ProductContext } from '../contexts/ProductContext';

const LowStockAlert = () => {

  const { products } = useContext(ProductContext);

  const lowStockItems = products.filter(product => product.quantity <= 5);

  return (

    <div className="low-stock-alert">

      {lowStockItems.length > 0 ? (

        <div className="alert">

          Low Stock Alert: {lowStockItems.map(item => item.name).join(', ')}

        </div>

        ) : null}

      </div>

    );

  };

export default LowStockAlert;

// ProductFilter.js

```

```

import React, { useContext } from 'react';

import { FilterContext } from '../contexts/FilterContext';

```



```
const ProductFilter = () => {

  const { filter, setFilter } = useContext(FilterContext);

  const categories = ['All', 'Category A', 'Category B', 'Category C'];

  return (

    <div className="product-filter">

      <label htmlFor="category">Filter by Category: </label>

      <select

        id="category"

        value={filter}

        onChange={(e) => setFilter(e.target.value)}

      >

        {categories.map((cat) => (

          <option key={cat} value={cat}>{cat}</option>

        ))}

      </select>

    </div>

  );

};

export default ProductFilter;
```

```
// FilterContext.js
```

```
import React, { createContext, useState } from 'react';
```

```
export const FilterContext = createContext();
```

```
export const FilterProvider = ({ children }) => {
```

```
  const [filter, setFilter] = useState('All');
```

```
  return (
```

```
    <FilterContext.Provider value={{ filter, setFilter }}>
```

```
      {children}
```

```
    </FilterContext.Provider>
```

```
  );
```

```
};
```

```
// ProductDetail.js
```

```
import React, { useContext, useState, useEffect } from 'react';
```

```
import { ProductContext } from '../contexts/ProductContext';
```

```
import { FilterContext } from '../contexts/FilterContext';
```

```
const ProductDetail = () => {
```

```
  const { selectedProduct, updateProduct } = useContext(ProductContext);
```

```
const { setFilter } = useContext(FilterContext);

const [editedProduct, setEditedProduct] = useState({});

useEffect(() => {

  if (selectedProduct) {

    setEditedProduct(selectedProduct);

  } else {

    setEditedProduct({ name: "", category: "", price: "", quantity: "" });

  }

}, [selectedProduct]);

const handleChange = (e) => {

  setEditedProduct({

    ...editedProduct,

    [e.target.name]: e.target.value

  });

};

const handleSave = () => {

  if (selectedProduct) {

    updateProduct(editedProduct);

  } else {

    updateProduct({ ...editedProduct, id: Date.now() }); // Add new

  }

}
```

```
};
```

```
return (
```

```
<div className="product-detail-container">
```

```
<h2>Product Detail</h2>
```

```
<label>
```

```
  Name:
```

```
<input
```

```
  name="name"
```

```
  value={editedProduct.name || ""}
```

```
  onChange={handleChange}
```

```
  aria-label="Name:"
```

```
</label>
```

```
<label>
```

```
  Category:
```

```
<input
```

```
  name="category"
```

```
  value={editedProduct.category || ""}
```

```
  onChange={handleChange}
```

```
  aria-label="Category:"
```

```
</label>
```

```
<label>
```

Price:

```
<input
```

```
  name="price"
```

```
  value={editedProduct.price || ""}
```

```
  onChange={handleChange}
```

```
  aria-label="Price:"
```

```
</label>
```

```
<label>
```

Quantity:

```
<input
```

```
  name="quantity"
```

```
  value={editedProduct.quantity || ""}
```

```
  onChange={handleChange}
```

```
  aria-label="Quantity:"
```

```
</label>
```

```
<button onClick={handleSave}>Save</button>
```

```
</div>
```

```
);
```

```
};
```

```
export default ProductDetail;
```